

Authoritative DNS, zone authoring and delegation

Follow-along: publish a domain from nothing, then hand part of it away

Overview

Every name you have ever looked up ended at a machine whose operator had written the reply into a file by hand. This lab is that job. You are given a domain, `uni.lab`, three servers that run `named` and hold nothing at all, and a parent zone that already tells the rest of the network to ask two of those servers about `uni.lab`. You write what they reply with: a zone file carrying the domain's addresses, its mail destination and one alias, a second server that keeps a copy in step with the first, a subdomain handed to a department's own server, and the records that turn each of this network's addresses back into a name.

The lab is about publishing names, and stops there. It does not sign them: DNSSEC is the subject of the DNS cache-poisoning lab, and nothing here is signed or validated. It does not configure a recursive resolver either, beyond using one as a measuring instrument, and it sends no forged packet at anything. What it does cover is the part a site operator owns: the file, the two servers that have to agree on it, the boundary where one operator's zone ends and another's begins, and the four ways a zone that looks right in an editor still replies wrongly on the wire.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab  # first time
cd ~/MiniLab && git pull                                     # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

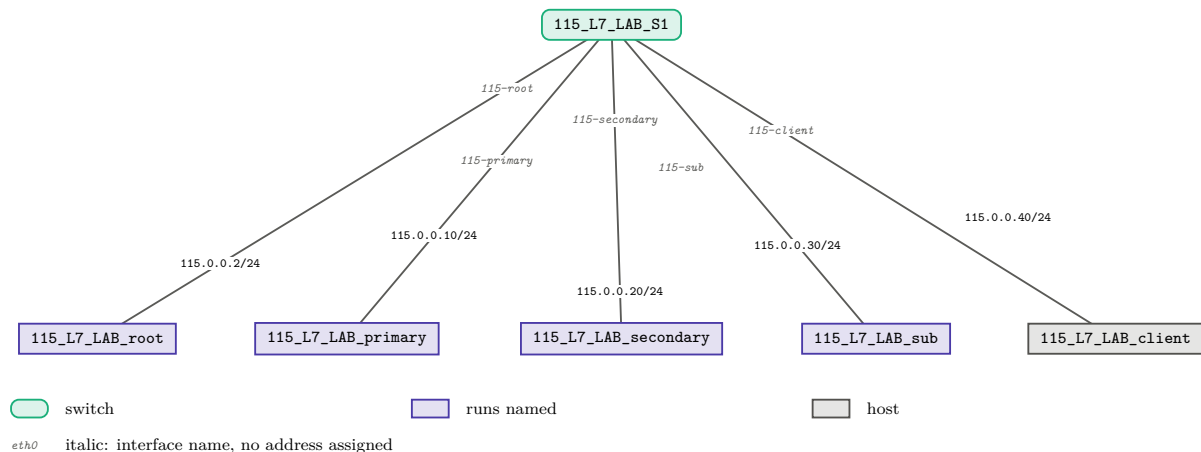
The root menu lists one category per row. Select **Host and service management**, then **Authoritative DNS, zone authoring and delegation**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: for every name and record type this lab asks you to publish, whether the right server answers with the right value. It takes over the whole screen, the arrow keys scroll its output, and Enter on **Done** closes it. Everything below the root server fails at first, and that is the state you start from.
- **Shell command for a node** lists every node (primary, secondary, sub, client, root) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.

- **Reset to baseline** deletes everything you wrote and puts the lab back to the state Spawn left it in, without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Six containers on one switched segment. Nothing here crosses a subnet boundary and there is no router, so every machine can reach every other machine from the moment Spawn finishes: anything that fails after that fails in the DNS and nowhere else.

Container	IPv4	IPv6	Role
115_L7_LAB_root	115.0.0.2	fd00:115::2	the root server, and everything above <code>uni.lab</code>
115_L7_LAB_primary	115.0.0.10	fd00:115::10	<code>ns1.uni.lab</code> ; Parts 1, 3 and 4 are written here
115_L7_LAB_secondary	115.0.0.20	fd00:115::20	<code>ns2.uni.lab</code> ; Part 2 is written here
115_L7_LAB_sub	115.0.0.30	fd00:115::30	<code>ns1.cs.uni.lab</code> ; Part 3 is written here too
115_L7_LAB_client	115.0.0.40	fd00:115::40	a recursive resolver; every query is sent from here
115_L7_LAB_S1	–	–	one Open vSwitch bridge, no STP, no VLANs

Two of those machines are given and are not yours to edit. The root server holds the three zones that sit above `uni.lab`, so the chain you extend is one a resolver really walks down. The client runs `named` as a recursive resolver whose entire configuration is a file naming the root server's address; it holds no zone, and it is how you check that what you published is what the rest of the network sees.

The other three arrive with `named` running, `/etc/bind/named.conf` written for you, and `/etc/bind/zones.conf` empty. That second file is included from the first, and it is where every zone you declare goes. Each server's log is `/var/log/named.log`.

Every machine carries these:

- `dig`, for sending one query and printing the whole reply.
- `named-checkzone`, which parses a zone file and reports what `named` would reject, without touching the running server.

- `named-checkconf`, the same for `named.conf` and the files it includes.
- `rndc`, which tells a running `named` to re-read its configuration or a zone file, and asks it what it currently holds.
- `curl`, `ping6`, `vi` and `nano`.

Assessment

There are seventeen questions, placed at the step each one is about. Answer all seventeen in a separate document; nothing is written in this handout.

Part 1: the zone file

Definition: Zone, and authoritative server

A **zone** is the part of the name tree one operator holds the records for, kept in one file. `uni.lab` is a zone; so are `lab` and the root above it. A server is **authoritative** for a zone when it replies out of its own copy of that file rather than by asking somebody else, and it says so in every reply by setting a bit in the header. A **recursive resolver** does the opposite job: it holds no zone, and resolves a question by asking authoritative servers in turn until it reaches one that has the record.

The machine at 115.0.0.10 is meant to be **authoritative** for `uni.lab`, and the parent zone above it has said so since the lab was spawned. Ask both machines the same question and compare what each one does with it. Every `dig` in this lab is run from the client, and `+norecurse` is what stops the server going and looking the name up somewhere else:

```
# 115_L7_LAB_client: run every dig in this lab from here
docker exec -it 115_L7_LAB_client bash
```

```
# 115_L7_LAB_client: ask the machine that is supposed to hold the zone
dig +norecurse @115.0.0.10 www.uni.lab A

# 115_L7_LAB_client: ask the root server the same question
dig +norecurse @115.0.0.2 www.uni.lab A
```

Question 1. Give the **status** each of those two replies carried, and the contents of the second reply's authority and additional sections. Then say what the root server is doing that the machine at 115.0.0.10 is not, in terms of what each one holds.

Definition: Resource record

One line of a zone file, and the unit the DNS moves around. It has four parts before its value: an **owner name** (which name it is about), a **TTL** (how many seconds anybody may keep it after fetching it), a **class** (always `IN` here), and a **type**. What follows is the **rdata**, whose shape depends on the type. In `www.uni.lab. 3600 IN A 115.0.0.10`, the owner name is `www.uni.lab.`, the TTL is 3600, the type is `A`, and the rdata is one IPv4 address.

A zone file is a list of **resource records** with two pieces of shorthand, and both of them bite. A name that does not end in a dot is not the name you wrote: the zone's own name is

appended to it, so inside `uni.lab`'s file, `www` means `www.uni.lab.` and `www.uni.lab` means `www.uni.lab.uni.lab.` A name that does end in a dot is taken exactly as written. The character `@` on its own stands for the zone's own name. Ahead of all of it, `$TTL` sets the TTL for every record that does not carry one of its own.

Definition: The SOA record

The Start of Authority record sits at a zone's apex and every zone has exactly one. Its rdata is seven fields in a fixed order: the name of the primary server, the responsible mailbox, and then five numbers. **Serial** is a version number the operator raises by hand on every edit. **Refresh** is how many seconds a secondary waits before checking for a new serial; **retry** is how long it waits after a check that failed; **expire** is how long it keeps replying for the zone when no check has succeeded, after which it must discard its copy (RFC 1034, section 4.3.5). **Minimum** was defined in RFC 1035 as a floor under every TTL in the zone, and redefined by RFC 2308 as the lifetime of a negative answer, which is what BIND 9.18 implements.

The responsible mailbox is written as a domain name rather than an address, so the `@` of the mailbox becomes a dot: `hostmaster.uni.lab.` is the name for `hostmaster@uni.lab.`

Write the zone file on the primary. Open a second terminal for it, because the `dig` session above stays useful throughout:

```
# 115_L7_LAB_primary: ns1.uni.lab, where the zone file goes
docker exec -it 115_L7_LAB_primary bash
vi /var/bind/uni.lab.zone
```

The **SOA record** comes first, then the records the zone has to carry. Use these five timer values, because later questions are asked about them: refresh 3600, retry 600, expire 1209600, minimum 300, and a serial in the `YYYYMMDDnn` form RFC 1912 recommends, so today's first edit is 2026083101.

```
$TTL 3600
@      IN  SOA ns1.uni.lab. hostmaster.uni.lab. (
                2026083101 ; serial
                3600      ; refresh
                600       ; retry
                1209600   ; expire
                300      ) ; minimum
```

Below it, publish these. The two `NS` records are what the zone itself says about which servers hold it, and they have to match what the parent already publishes, which you read out of the second reply in Question 1.

Name	Type	Value
uni.lab	NS	ns1.uni.lab.
uni.lab	NS	ns2.uni.lab.
uni.lab	MX	preference 10, mail.uni.lab.
ns1.uni.lab	A	115.0.0.10
ns2.uni.lab	A	115.0.0.20
mail.uni.lab	A	115.0.0.20
www.uni.lab	A	115.0.0.10
www.uni.lab	AAAA	fd00:115::10
client.uni.lab	A	115.0.0.40
web.uni.lab	CNAME	www.uni.lab.

Definition: A, AAAA, MX and CNAME

A carries one IPv4 address and AAAA one IPv6 address (RFC 3596); a name may have both, and a client picks between them. MX carries a preference number and the name of a host that accepts mail for the zone, lowest preference first. CNAME carries one name and means “this name is an alias for that one”: a resolver that finds a CNAME restarts its lookup at the name in the rdata, so `web.uni.lab` ends up resolving to whatever `www.uni.lab` resolves to.

The **MX** record names `mail.uni.lab`, and `mail` is given an **A** record of its own rather than being made an alias. RFC 2181, section 10.3, is why: the name used in an **NS** record, or in the host half of an **MX** record, must not be an alias, and must have address records directly. A server searching for an **MX** appends the address records for the name it found; it does not follow a **CNAME** to do so, so an alias there costs an extra query on every piece of mail.

Check the file before you make `named` read it. `named-checkzone` takes the zone’s name and the file, parses it exactly as `named` would, and exits non-zero on anything `named` would refuse:

```
# 115_L7_LAB_primary
named-checkzone uni.lab /var/bind/uni.lab.zone
```

Then declare the zone. `/etc/bind/zones.conf` is included from `named.conf` and is empty; a zone needs both a statement here and the file it names, and either one without the other leaves `named` holding nothing:

```
# 115_L7_LAB_primary
vi /etc/bind/zones.conf
```

```
zone "uni.lab" {
    type primary;
    file "/var/bind/uni.lab.zone";
};
```

Definition: rndc

The control channel of a running **named**, over a loopback connection authenticated with a shared key. **rndc reconfig** makes **named** re-read **named.conf** and pick up zones that were added or removed; **rndc reload** makes it re-read the zone files themselves; **rndc zonestatus <zone>** reports what it currently holds for one zone, and fails when it holds none.

```
# 115_L7_LAB_primary
rndc reconfig
rndc zonestatus uni.lab
```

Definition: The AA bit

One bit in the header of every DNS reply, printed by **dig** in the **flags:** line as **aa**. A server sets it when the answer came out of a zone it holds. A reply from a resolver never carries it, however correct the answer is, because the resolver copied that answer from somewhere else.

Now ask the same question two ways: straight at the primary, and through the resolver on the client, which starts at the root and walks down.

```
# 115_L7_LAB_client
dig +norecurse @115.0.0.10 www.uni.lab A
dig @115.0.0.40 www.uni.lab A
```

Question 2. Give the **flags:** line from each of those two replies in full. Both carry the same address, so name the flag that differs and say what that flag tells you about where each answer was read from.

Question 3. Your SOA's second field is a domain name standing in for a mailbox. Give the value you wrote and the mailbox address it encodes. Then apply the same encoding to a mailbox whose local part is **dns.admin**, and say what goes wrong when it is applied naively.

Question 4. Your CNAME line ends in a dot. Give the name **web.uni.lab** would have become an alias for had you left that dot off, and say which piece of zone-file shorthand produces it.

Definition: Negative answer, and NXDOMAIN

When a server authoritative for a zone is asked about a name inside that zone that does not exist, it replies with the status **NXDOMAIN** and an empty answer section. RFC 2308, section 3, requires it to put the zone's SOA record into the *authority* section of that reply, because there is no record in the answer section for a TTL to ride on, and something has to say how long the absence may be remembered.

Ask for a name your zone does not hold:

```
# 115_L7_LAB_client
dig +norecurse @115.0.0.10 nowhere.uni.lab A
```

Question 5. Give the status, the number of records in the answer section, and the section the SOA came back in with its TTL. Name the SOA field that TTL was taken from, and say why

it is not the zone's `$TTL` of 3600.

The last thing in this part is a rule the file format enforces. RFC 2181, section 10.1, allows a name to hold one CNAME record or one or more records of other types, and never both, because a resolver that found a CNAME is told to restart its lookup elsewhere and cannot also be given an answer here. `named-checkzone` refuses a file that breaks it.

Question 6. You are about to add `web IN A 115.0.0.10` to a copy of your zone file, beside the CNAME already at that name. Before running anything, predict what `named-checkzone` will do with it and what it will say. Then run the two commands below and quote the message it printed and the exit status it returned.

```
# 115_L7_LAB_primary
cp /var/bind/uni.lab.zone /tmp/broken.zone
printf 'web      IN      A      115.0.0.10\n' >> /tmp/broken.zone
named-checkzone uni.lab /tmp/broken.zone ; echo "exit status: $?"
```

Part 2: a second server that keeps a copy

One server holding the only copy of a zone is one machine away from the domain going dark. The fix is a second server that holds the same records and is authoritative for them in the same way, which means it needs the file, and needs it to change when the first one's file changes. Nobody edits it: the second server fetches the whole zone over TCP from the first, and refetches it whenever the first says the zone has moved on.

Definition: Zone transfer (AXFR), and the serial

A **zone transfer** is a request of type **AXFR**, served over TCP with every record in the zone, the apex SOA first and the same SOA again last (RFC 1034, section 4.3.5). What decides whether a secondary asks for one is the **serial**: it fetches only the SOA, compares that serial with the one in its own copy, and transfers only when the primary's is higher. A record changed without the serial being raised is a change the secondary is entitled to ignore.

Before restricting anything, see what a stranger can take. The client is not a name server, holds no copy of anything, and is not named in the zone:

```
# 115_L7_LAB_client
dig @115.0.0.10 uni.lab AXFR
```

Question 7. Count the records the transfer printed and give the number. The SOA appears twice in that list; say which position each copy is in and what the second one is for.

BIND 9.18's default is to allow a **zone transfer** to any host that asks, so the line you are about to write removes access rather than granting it. Add it to the zone's statement on the primary, naming the one machine entitled to a copy:

```
# 115_L7_LAB_primary
vi /etc/bind/zones.conf
```

```
zone "uni.lab" {
    type primary;
    file "/var/bind/uni.lab.zone";
    allow-transfer { 115.0.0.20; };
};
```

Raise the serial to 2026083102 while the file is open, so the transfer that follows has something to carry, then make **named** pick up both changes:

```
# 115_L7_LAB_primary
vi /var/bind/uni.lab.zone      # serial -> 2026083102
rndc reconfig && rndc reload
```

Now the secondary. Its statement names the machine to fetch from and a file to keep the copy in; **named** writes that file itself, so nothing creates it:

```
# 115_L7_LAB_secondary: ns2.uni.lab
docker exec -it 115_L7_LAB_secondary bash
vi /etc/bind/zones.conf
```

```
zone "uni.lab" {
    type secondary;
    primaries { 115.0.0.10; };
    file "/var/bind/secondary/uni.lab.zone";
};
```

```
# 115_L7_LAB_secondary
rndc reconfig
rndc zonestatus uni.lab
tail -5 /var/log/named.log
```

Question 8. Repeat the AXFR request from Question 7, first against 115.0.0.10 and then against 115.0.0.20. Give what each one returned. Then name the field of the incoming request the primary compared against its **allow-transfer** list, and say why the secondary serving the request at all is not a hole in the restriction you just wrote.

Definition: NOTIFY

A message a primary sends the moment a zone's serial changes, so a secondary finds out in seconds rather than waiting out its refresh interval (RFC 1996). Its default recipients are not configured anywhere: they are every server named in the zone's own NS records, minus whichever one the SOA's first field names.

```
# 115_L7_LAB_primary
grep notify /var/log/named.log
```

Question 9. You never listed a machine to notify. Name the records **named** read to work out who to send a **NOTIFY** to, give the machines that puts on the list for **uni.lab**, and name the

one server in your zone's NS set that is not on it and why.

The primary re-reads its zone file whenever `rndc reload` finds the file has changed, whatever the serial says. The secondary does not work that way, and the next edit shows the gap between the two.

```
# 115_L7_LAB_primary: change www's address, leave the serial at 2026083102
vi /var/bind/uni.lab.zone      # www IN A -> 115.0.0.99
rndc reload
```

```
# 115_L7_LAB_client
dig +norecurse @115.0.0.10 www.uni.lab A
dig +norecurse @115.0.0.20 www.uni.lab A
dig +norecurse @115.0.0.10 uni.lab SOA
dig +norecurse @115.0.0.20 uni.lab SOA
```

Question 10. Give the address each server returned for `www.uni.lab` and the serial each one reported. Then say which of the SOA's five numbers would eventually have carried the change across anyway, and how many seconds it would have taken at worst, using the value you wrote.

Put the address back to 115.0.0.10 and reload, so the two copies agree again before Part 3.

Question 11. The primary in this lab is one container and can be stopped. Suppose it stops replying entirely and stays that way. Using your own SOA values, give how long the secondary keeps replying to questions about `uni.lab` and what it does with a question about the zone once that runs out.

Part 3: handing a subdomain to somebody else

The computer science department wants to run its own names under `cs.uni.lab`, on its own server, without you approving every record. The DNS is built for exactly this: a zone can stop at any point in the tree and say that everything below that point is somebody else's. The parent keeps no records from below the cut, and replies to a question about a name down there by naming the servers that do.

Definition: Delegation, zone cut and referral

A **delegation** is one or more NS records in the parent zone at the name being handed over. The name they sit at is the **zone cut**: above it the parent is authoritative, below it the child is. Asked about a name below the cut, the parent replies with no answer records, the child's NS records in the authority section, and the AA bit clear, because the reply is not an answer out of its own zone. That reply is a **referral**, and it means "ask these servers instead".

Write the child zone first. A **delegation** pointing at a server that holds no such zone is worse than no delegation at all, because the parent has already told the resolver to stop asking anywhere else:

```
# 115_L7_LAB_sub: ns1.cs.uni.lab
docker exec -it 115_L7_LAB_sub bash
vi /var/bind/cs.uni.lab.zone
```

```
$TTL 3600
@      IN  SOA  ns1.cs.uni.lab. hostmaster.cs.uni.lab. (
                2026083101  ; serial
                3600        ; refresh
                600         ; retry
                1209600     ; expire
                300 )       ; minimum

@      IN  NS   ns1.cs.uni.lab.

ns1     IN  A    115.0.0.30
www     IN  A    115.0.0.30
```

```
# 115_L7_LAB_sub
named-checkzone cs.uni.lab /var/bind/cs.uni.lab.zone
vi /etc/bind/zones.conf      # a "zone cs.uni.lab" statement, type primary, naming \
    that file
rndc reconfig && rndc zonestatus cs.uni.lab
```

Definition: Glue

An address record a parent zone carries for a name that belongs to the child, put there so a referral can be followed. It is needed exactly when the name server's name sits inside the zone being delegated: RFC 1034, section 4.2.1, puts it as being told that to learn a server's address you should contact the server at the address you wish to learn. RFC 8499, section 7, calls such a server **in-domain** and states plainly that an in-domain name server name needs glue records or name resolution fails.

You already have a worked example of it, in the reply Question 1 asked you to read: the root server delegates `uni.lab` to `ns1.uni.lab` and `ns2.uni.lab`, and because both of those names are inside `uni.lab`, it carries both their addresses in the additional section. The delegation you are about to write has the same shape, so it needs the same thing. Append it to the parent zone and raise the serial:

```
# 115_L7_LAB_primary
vi /var/bind/uni.lab.zone      # serial -> 2026083103, and add the two lines below
```

```
cs      IN  NS   ns1.cs.uni.lab.
ns1.cs  IN  A    115.0.0.30
```

```
# 115_L7_LAB_primary
named-checkzone uni.lab /var/bind/uni.lab.zone
rndc reload
```

One line of that output needs saying before it worries you. With the delegation in the file, `named-checkzone` prints `zone uni.lab/IN: getaddrinfo(ns1.cs.uni.lab) failed: Try again`. It is trying to look the delegated server's name up through the container's own stub resolver, which is configured with no name server and has never heard of this lab's tree. That says nothing about the file: the last line is still OK and the exit status is still 0.

Definition: The EDNS OPT record

A pseudo-record `dig` adds to every query it sends and that a server returns, carrying the largest reply size the sender will accept and a few flags (RFC 6891). It rides in the additional section, so it is counted in the `ADDITIONAL:` number of every reply in this lab. A referral carrying one `glue` address therefore reports `ADDITIONAL: 2`, not 1.

```
# 115_L7_LAB_client
dig +norecurse @115.0.0.10 www.cs.uni.lab A
```

Question 12. Give the status, the three section counts, and the `flags:` line from that reply, and say which of the additional records is the glue. The primary is authoritative for `uni.lab`, and `www.cs.uni.lab` is a name underneath it; explain why the reply nonetheless does not carry the `aa` flag.

```
# 115_L7_LAB_client
dig +norecurse @115.0.0.10 cs.uni.lab SOA
dig +norecurse @115.0.0.30 cs.uni.lab SOA
```

Question 13. Give the answer count and the `flags:` line each of those two replies carried. Both machines hold NS records for `cs.uni.lab`; say what each machine's copy of those NS records is for, and which of the two holds the child zone's apex.

Now the whole chain, from a resolver that was told nothing except where the root is. Empty its cache first, so nothing it learned earlier replies for it:

```
# 115_L7_LAB_client
rndc flush
dig +trace www.cs.uni.lab A
curl http://www.cs.uni.lab/
```

`+trace` makes `dig` do the walk itself and print each step: the server it asked, and the records that came back. One thing about the output needs saying, because nothing so far predicts it. The root container is authoritative for the root zone *and* for `lab`, being one machine holding both sides of that cut, so it replies straight out of the deeper of the two zones and `lab` never appears as a step of its own.

Question 14. Give the servers `+trace` asked, in the order it asked them, and the zone each one referred it to next. Then give what `curl` printed, and name the machine that produced it.

The delegation rests on two records and only one of them is obvious. Delete the glue, raise the serial so the secondary follows the change too, and look again:

```
# 115_L7_LAB_primary
vi /var/bind/uni.lab.zone      # delete the "ns1.cs IN A" line, serial -> 2026083104
rndc reload
```

Question 15. Before running anything: the NS record is still there and `ns1.cs.uni.lab` is still the right name, so predict whether the resolver on the client can still reach `www.cs.uni.lab`,

and give your reasoning in terms of the two names involved. Then run the three commands below and give the additional count of the referral and the status the resolver returned.

```
# 115_L7_LAB_client
dig +norecurse @115.0.0.10 www.cs.uni.lab A
rndc flush
dig @115.0.0.40 www.cs.uni.lab A
```

Put the line back, raise the serial to 2026083105, reload, and check the resolver reaches the name again before going on.

Part 4: turning the addresses back into names

A lookup from a name to an address goes down the tree from the root through `lab` to `uni.lab`. Going the other way needs a tree as well, and an address is not a name, so the DNS builds one out of it: the four octets written backwards under the domain `in-addr.arpa`. `115.0.0.10` becomes `10.0.0.115.in-addr.arpa`, which puts every address in `115.0.0.0/24` under one name, `0.0.115.in-addr.arpa`, that can be a zone of its own and delegated like any other. Reversing the octets is what makes that work: an IPv4 address is most general on the left and the DNS is most general on the right.

Definition: PTR

A record whose owner name is an address written in the `in-addr.arpa` form and whose rdata is one domain name. It is the reverse tree's only interesting record type, and `dig -x` is shorthand that builds the owner name for you and asks for the PTR at it.

The root server already delegates `0.0.115.in-addr.arpa` to `ns1.uni.lab`, so as with `uni.lab` in Part 1, the rest of the network is being sent to a machine that has nothing to say. Write the zone:

```
# 115_L7_LAB_primary
vi /var/bind/0.0.115.in-addr.arpa.zone
```

Give it the same SOA shape as the forward zone, an NS record naming `ns1.uni.lab.`, and a **PTR** for each of these five addresses, pointing at the name that machine is known by: `115.0.0.2` is `ns.root-lab.`, `.10` is `ns1.uni.lab.`, `.20` is `ns2.uni.lab.`, `.30` is `ns1.cs.uni.lab.` and `.40` is `client.uni.lab.` Owner names here follow the same shorthand as any other zone file, so the last octet alone is enough. Then declare it beside the forward zone in `/etc/bind/zones.conf`, check it, and reload.

```
# 115_L7_LAB_client
rndc flush
dig -x 115.0.0.30
dig -x 115.0.0.10
```

Question 16. Give the owner name the first of those two commands asked about, taken from the question section of the reply. Then, without running anything, give the owner name a reverse lookup of `115.0.0.7` would ask about, and the zone this lab's servers would answer it out of.

The two trees are separate zones, held in separate files, and nothing checks that they agree. Break the agreement on purpose and look both ways:

```
# 115_L7_LAB_primary
vi /var/bind/0.0.115.in-addr.arpa.zone    # the PTR for .40 -> ghost.uni.lab.
rndc reload
```

```
# 115_L7_LAB_client
rndc flush
dig -x 115.0.0.40 +short
dig @115.0.0.40 ghost.uni.lab A
```

Question 17. Give what each of those two commands returned. A log file that records a hostname instead of an address gets that hostname from a lookup exactly like the first one; say what this pair of results means for anybody reading such a log, and name the second lookup an operator would have to do to catch it.

Put the PTR back to `client.uni.lab.` and reload.

Checks: what "done" means

Run Status. Every line is a query it sends and the value it expected back, and the four groups match the four parts:

- **Part 1** the primary replies with the `aa` bit set for all ten records in the forward zone, and denies `nothere.uni.lab` authoritatively with the SOA behind it.
- **Part 2** the secondary is loaded, both copies report the same serial, the secondary replies for every record out of its own copy, and a transfer request from the client is refused.
- **Part 3** the primary refers `www.cs.uni.lab` downwards with the child server's address in the additional section, the child server replies for its own zone, and the resolver on the client reaches the name starting from the root.
- **Part 4** `dig -x` returns a name for each of the four addresses.

The last three lines are the whole chain in one piece: `http://www.uni.lab/` and `http://www.cs.uni.lab/` each reach the machine their records name, and the address in your AAAA record replies to an echo request. When a check fails, Status prints the tail of each server's log underneath, which is where a zone file `named` refused to load says so.

Safety note

Everything in this lab runs on one Open vSwitch bridge inside containers, with no route off the host. The zone `uni.lab`, the TLD `lab` and the root zone here exist only on this bridge: the root server is authoritative for a root zone of its own, and the client resolver is pointed at it and at nothing else, so no query in this lab leaves the machine and none of these names is visible to anything outside it. The reverse zone covers `115.0.0.0/24`, which is used here as lab addressing on an isolated segment.

Teardown

When you have answered the questions, remove the lab: pick **Teardown** in the TUI, or run `scripts/teardown.sh` directly. It deletes this lab's six containers and nothing else.